

결제 한도 차감 + 수수료 정책

동시에 몰리는 결제 요청 속에서도, 한도와 수수료를 한 치의 오차 없이 계산하는 서버를 완성합니다.

💡 이 문서는 요약본입니다. 처음 과제를 시작하시는 분들을 위해 큰 그림만 먼저 보여드려요. 자세한 API 규칙, 수수료 규칙 표, 문제 6개, 힌트는 저장소의 README.md 에 전부 정리되어 있습니다. 이미 뼈대 프로젝트가 준비되어 있으니, 처음부터 다 만들 필요는 없습니다 — 빠진 부분만 채우면 돼요.

6개

풀어야 할 문제

구현 2개 + 테스트 2개 + 서술 2개로 구성됩니다

12개

점검 시나리오

이 시나리오가 모두 통과하면 과제 완성입니다

2곳

손봐야 할 부분

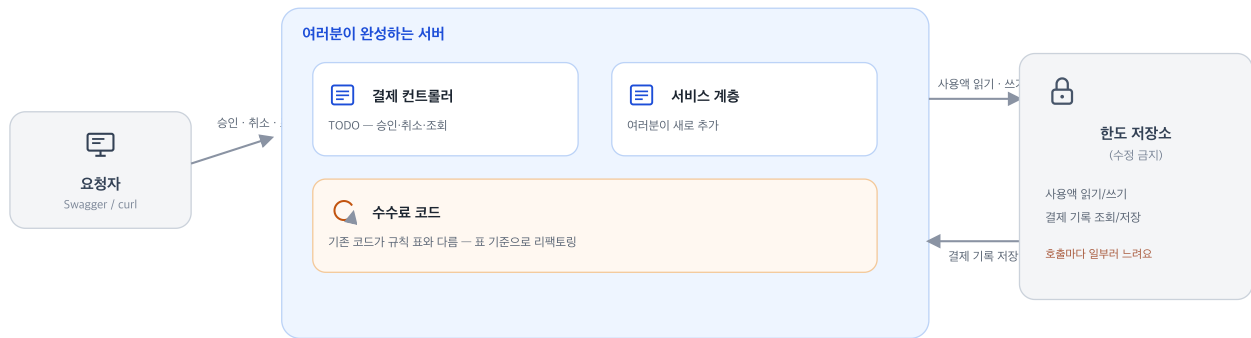
동시성 처리(새로 작성) + 수수료 코드(리팩토링)

제한 없음

소요 시간

빠르게보다 정확하게 만드는 것이 목표예요

전체 구조 — 이미 준비된 것과 채워야 할 것



파란색은 여러분이 새로 구현하는 부분, 주황색은 리팩토링 대상, 회색은 손댈 수 없는 저장소입니다 — 저장소가 느린 사이에 정확성을 지키는 것이 핵심입니다.

이 과제에서 이루려는 것 / 신경 쓰지 않아도 되는 것

🎯 목표 Outcome

- 동시 요청에도 **정확히 1건만** 승인·취소되게 만듭니다
- 한도와 사용액이 **언제나 정확**해야 합니다
- 수수료 규칙표를 **경계값까지** 정확히 구현합니다
- 동시성 규칙을 **테스트로 증명**하고, 이유를 글로 설명합니다

🧑‍💻 신경 쓰지 않아도 되는 것 Not in scope

- DB·JPA 사용 (이 과제는 메모리 저장소만 씁니다)
- 실제 카드사·PG 연동 (범위 밖)
- 여러 대의 서버로 확장하는 것 (서버 1대 가정)
- 날짜별 한도 초기화 (다루지 않습니다)

제공되는 것 — 한도 저장소

이미 완성되어 있고, 여러분이 수정할 수 없는 클래스입니다. 다만 아래 세 가지 특징을 일부러 가지고 있어요 — 실무의 외부 저장소가 흔히 그렇듯이요.

일부러 느려요

메서드를 호출할 때마다 지연이 있습니다.
"확인하고 저장"하는 틈이 크게 벌어집니다.

수정 금지예요

메서드 시그니처와 동작을 바꿀 수 없습니다.
안전하게 쓰는 건 호출하는 쪽의 몫입니다.

경합을 유도해요

여러 요청이 같은 사용자·같은 결제 ID를 동시에 건드리는 상황을 그대로 재현합니다.

가장 중요한 함정 — "확인하고 실행하는" 사이에 끼어들기

"사용액을 읽어 한도가 남았는지 확인하고, 남았으면 차감한다" — 이 세 단계 사이에 다른 요청이 끼어들면 둘 다 통과해서 한도를 넘길 수 있습니다. 결제 ID 중복 방지도 똑같은 함정을 가지고 있습니다.

① 확인

요청 A가 사용액을 읽어 한도가 남았는지 확인합니다



② 끼어들

저장소가 느린 사이, 요청 B도 같은 사용액을 읽어 "남았다"고 판단합니다



③ 충돌

A, B 둘 다 승인되어 한도를 넘겨버립니다



④ 방지

같은 대상(사용자·결제 ID)에 대한 확인·차감을 한 번에 하나씩만 지나가게 만듭니다

락의 범위를 어떻게 잡을지는 README.md 12장 힌트 2에서 다시 다룹니다.

약속 요약 — 이것만은 형식을 지켜주세요

항목	규칙
결제 승인	POST /api/v1/payments — 성공 200, ID 중복 409, 한도 초과 422
결제 취소	POST /api/v1/payments/{id}/cancel — 성공 200, 없음 404, 이미 취소 409
한도 조회	GET /api/v1/users/{userId}/limit — 사용자당 한도는 1,000,000원 고정
수수료 계산	GET /api/v1/fees — 승인 API의 fee 와 반드시 같은 값
수수료 규칙	1,000원 미만·VIP 국내 면제, 등급·국내외별 요율, 고액 감면, 절사 후 최소 100원, 최대 50,000원

점검 시나리오 — 이 12개가 통과하면 완성입니다

① 정상 동작 · 경계값 (S1,S2,S6~S8,S10)

6개

- 정상 승인 → 수수료·잔여 한도 정확
- 같은 ID 재요청(순차) → 409
- 한도 경계(1,000,000원)까지 정확히 승인
- 입력 오류 → 400
- 정상 취소 / 없는 결제 취소
- 취소한 ID로 다시 승인 → 409

② 동시 요청 안전성 (S3,S4,S5,S9,S11)

5개

- 같은 ID 동시 요청 → 정확히 1건만 성공
- 다른 사용자가 같은 ID로 동시 요청 → 1건만
- 한도를 넘는 동시 요청 → 한도 안에서만 승인
- 같은 결제의 취소가 동시에 여러 건 → 1건만
- 승인·취소가 뒤섞여도 최종 사용액 정확

③ 수수료 규칙 (S12)

1개

- 모든 입력(경계값 포함)에서 계산 결과가 규칙 표와 정확히 일치

처음이라면, 이렇게 시작해 보세요

1 이 문서로 큰 그림을 먼저 파악하세요.

무엇이 이미 준비돼 있고, 무엇을 채워야 하는지 확인했다면 절반은 준비된 거예요.

2 README.md를 처음부터 끝까지 읽으세요.

특히 5장(API)과 6장(수수료 규칙 표)은 몇 번씩 다시 찾아보게 될 거예요.

3 `./gradlew bootRun` 으로 뼈대부터 띄워보세요.

정상 승인(S1) 하나가 되는 걸 먼저 확인한 뒤 시나리오를 눌러보세요.

4 문제 1(동시성)부터 순서대로 풀어보세요.

구현 후 바로 문제 2(테스트)로 스스로 증명하는 습관을 들이면 좋아요.

5 막히면 README 12장의 힌트를 하나씩만 열어보세요.

정답을 알려주진 않지만, 어느 방향으로 생각해야 할지 짚어줍니다.

🙌 정답이 하나로 정해진 과제가 아닙니다. 왜 그렇게 설계했는지 설명할 수 있다면, 그것으로 충분해요.

자세한 내용은 저장소의 README.md를 확인해 주세요. 화이팅입니다!